functional components and the corresponding lifecycle methods in class components, illustrating how to achieve similar behavior in both paradigms.

Table A.11 Summary of lifecycle phases in React

Lifecycle phase	Functional component Hook	Class component method
Mounting	`useEffect` body	`componentDidMount`
Updating	`useEffect` body (with dependencies)	`componentDidUpdate`
Unmounting	Cleanup function returned by `useEffect`	`componentWillUnmount`

`useEffect` is essential for managing side effects in functional React components. Side effects encompass any operations that interact with the outside world, modify the DOM directly, or perform actions outside React's normal rendering flow. In practice, useEffect is commonly used for tasks such as fetching data from an API when a component mounts, setting up and tearing down event listeners (e.g., listening for window resizes), managing subscriptions to external data sources (like `WebSockets`), or directly manipulating the DOM when absolutely necessary.

To ensure `useEffect` calls contribute to performance and maintain application stability, it is critical to follow several best practices. Defining the correct dependencies for each `useEffect` hook is essential—every variable or function used within the effect should be included in the dependency array. This ensures the effect runs only when those specific dependencies change, preventing unnecessary re-executions and potential performance bottlenecks. If a function used within the effect does not change between renders, wrapping it with `useCallback` maintains a stable reference and prevents the effect from re-running unnecessarily. Splitting `useEffect` hooks into smaller, more focused units is also recommended. Each hook should handle a specific task, resulting in cleaner, more modular, and easier-to-debug code. For example, one `useEffect` might handle data fetching, while another manages event listeners.